

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

MÔN: MẠNG MÁY TÍNH

Lương Hưng Phát – 24127298

Nguyễn Thành Đạt – 24127021

Nguyễn Anh Khôi – 24127430

ĐỒ ÁN 2

Phân tích Gói tin Mạng với Wireshark



fit@hcmus

GIẢNG VIÊN HƯỚNG DẪN

Cô CHUNG THÙY LINH

Cô HUỲNH THỤY BẢO TRÂN

TP. Hồ Chí Minh, tháng 7/2025

Mục lục

Mục lục	I
1 PHÂN TÍCH LƯU LƯỢNG HTTP	3
1.1 Định vị gói tin HTTP	3
1.1.1 HTTP GET	3
1.1.2 HTTP 200 OK	3
1.2 Phân biệt HTTP & HTTPS	4
1.3 Kết nối Duy trì & Không Duy trì	6
1.3.0.1 Phân tích loại kết nối HTTP	6
1.3.1 Lợi thế của kết nối HTTP bền vững	8
1.4 Transfer-Encoding: Chunked	8
1.4.1 Kiểm tra tiêu đề Transfer-Encoding	8
1.4.2 Lợi ích của Transfer-Encoding: chunked	9
2 PHÂN TÍCH LƯU LƯỢNG DNS (IPv4)	11
2.1 Tổng quan	11
2.2 Gói tin truy vấn DNS: “Loại truy vấn” và “Tên truy vấn” trong gói tin	12
2.2.1 Thông tin gói tin	12
2.2.2 Chức năng	12
2.3 Gói tin phản hồi DNS tương ứng và hiển thị phần “Answer”	13
2.3.1 Thông tin gói tin	13
2.3.2 Chức năng	13
2.4 Giao thức tầng vận chuyển được DNS sử dụng chủ yếu	14
2.4.1 Giao thức	14
2.4.2 Lý do	14
2.4.3 Ngoại lệ khi dùng TCP	14
2.5 Truy vấn DNS cho trang web đã truy cập	15
2.5.1 Quan sát	15
2.5.2 Các nguyên nhân có thể	15
2.5.3 Khi nào sẽ thấy truy vấn example.com	15

3

TÌM HIỂU GIAO THỨC TCP & UDP

16

3.1

So sánh giao thức TCP và UDP

16

3.2

So sánh TCP và UDP

16

3.3

Lưu lượng TCP

17

3.3.1

Phân tích quá trình bắt tay TCP (TCP Handshake)

17

3.3.2

Phân tích quá trình ngắt kết nối TCP (TCP Teardown)

19

3.3.3

Các cổng được sử dụng

20

3.3.4

Giải thích Window size value và Calculated window size

20

3.4

Lưu lượng UDP

22

3.4.1

Gói tin UDP có cơ chế bắt tay tương tự TCP không?

22

3.4.2

Các thông tin trong header UDP và so sánh với header TCP

23

3.4.3

Các loại ứng dụng và dịch vụ sử dụng UDP

25

4

GIẢI QUYẾT VẤN ĐỀ GIẢ ĐỊNH

27

4.1

TẠO MÔI TRƯỜNG GIẢ ĐỊNH

27

4.2

CHẨN ĐOÁN NGUYÊN NHÂN GÂY LỖI

27

4.3

NHỮNG GIAO THỨC VÀ GÓI CẦN XEM XÉT

28

4.4

XÁC ĐỊNH CỤ THỂ NGUYÊN NHÂN GÂY LỖI

28

4.4.1

DNS

28

4.4.2

TCP

28

4.4.3

Nguyên nhân cốt lõi

31

4.4.4

Tìm và khắc phục lỗi

31

4.4.4.1

Xác định IP

31

4.4.4.2

Tìm nguồn RST / Nơi drop gói

32

4.4.4.3

Kiểm tra TCP connect

32

Danh sách hình vẽ

- 1.1 Nội dung tiêu đề Host 3
- 1.2 Nội dung tiêu đề Content-Type 4
- 1.3 Minh chứng HTTP qua Protocol & PortTCP 5
- 1.4 Minh chứng HTTP qua GET & RESPONE 5
- 1.5 Minh chứng HTTP qua nội dung gói tin 6
- 1.6 Quá trình TCP 7
- 1.7 Connection: keep alive 7
- 1.8 Thực hiện TCP hand-shake 8
- 1.9 Transfer-Encoding 9

- 2.1 Gói tin DNS Query và Response cho nasa.gov 11
- 2.2 Gói tin DNS Query cho nasa.gov 12
- 2.3 Gói tin DNS Response cho nasa.gov 13
- 2.4 Toàn bộ gói tin trong capture DNS 15

- 3.1 TCP HandShake 18
- 3.2 FIN, ACK 19
- 3.3 Cổng nguồn - Cổng đích đến 20
- 3.4 Window Size 20
- 3.5 Toàn bộ gói tin trong capture UDP 22
- 3.6 Header của gói tin UDP 23
- 3.7 Header của gói tin TCP 24
- 3.8 Các Destination Port khi bắt gói tin UDP 25

- 4.1 Filter DNS 28
- 4.2 SYN From Client 29
- 4.3 SYN và ACK 29
- 4.4 Retransmission 30
- 4.5 Reset 31

Danh sách bảng

3.1 So sánh TCP và UDP 17

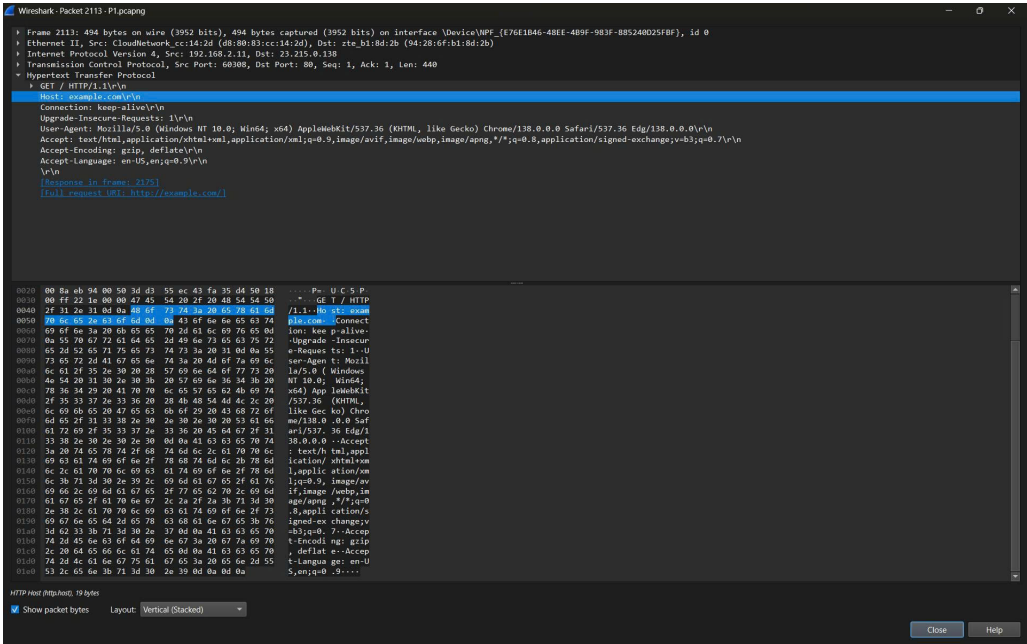
Phần 1

PHÂN TÍCH LƯU LƯỢNG HTTP

1.1 Định vị gói tin HTTP

1.1.1 HTTP GET

Trong một yêu cầu HTTP GET, trường **Host** trong tiêu đề chứa tên miền (domain name) của máy chủ mà yêu cầu đang được gửi đến. Giúp xác định máy chủ mà máy khách (trình duyệt web) muốn liên lạc.

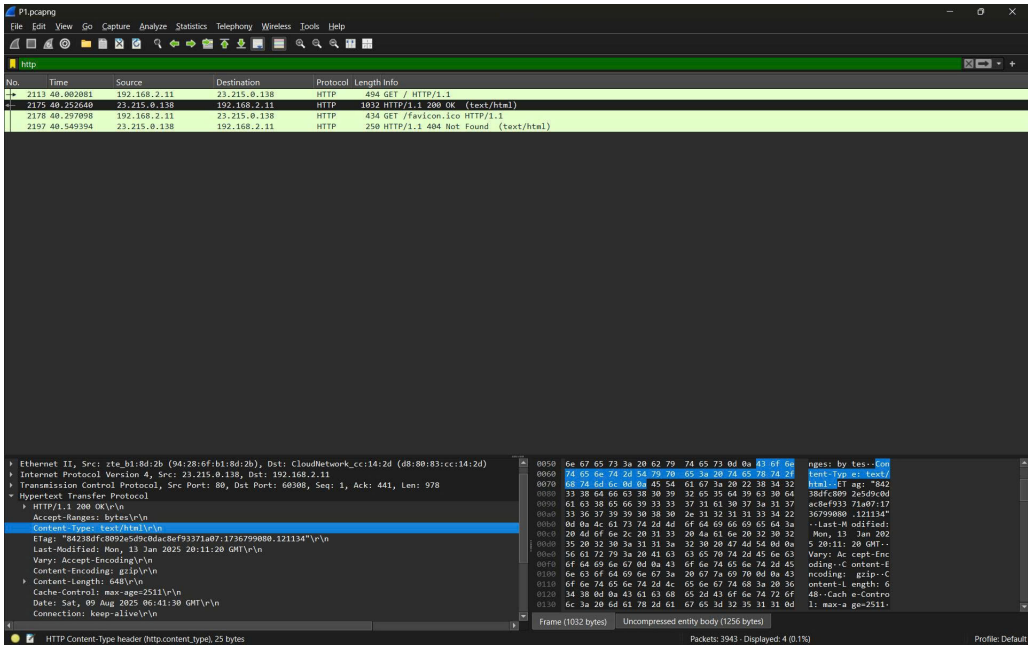


Hình 1.1: Nội dung tiêu đề Host

Trong quá trình bắt gói tin này, yêu cầu HTTP GET trả về tiêu đề `Host: example.com`, đây chính là trang web đang truy cập.

1.1.2 HTTP 200 OK

Phản hồi có mã trạng thái `200 OK` là một phản hồi thành công, cho biết yêu cầu của máy khách đã được máy chủ xử lý thành công.



Hình 1.2: Nội dung tiêu đề Content-Type

- **Số hiệu gói tin:** Gói tin số 1832 trong bản chụp hiển thị một phản hồi HTTP có mã trạng thái 200 OK.
- **Thông tin gói tin:** Gói tin này cho thấy máy chủ (192.168.2.118) đang phản hồi lại máy khách (10.254.51.59) với thông báo HTTP/1.1 200 OK.

Trong quá trình bắt gói tin này, trường Content-Type trả về text/html. Đây là kiểu phương tiện (media type). Nó chỉ định rằng dữ liệu trong phần nội dung của phản hồi là một tài liệu văn bản được định dạng bằng HTML.

Thông tin này rất quan trọng vì nó cho trình duyệt web biết cách xử lý và hiển thị dữ liệu nhận được (hiển thị dưới dạng một trang web).

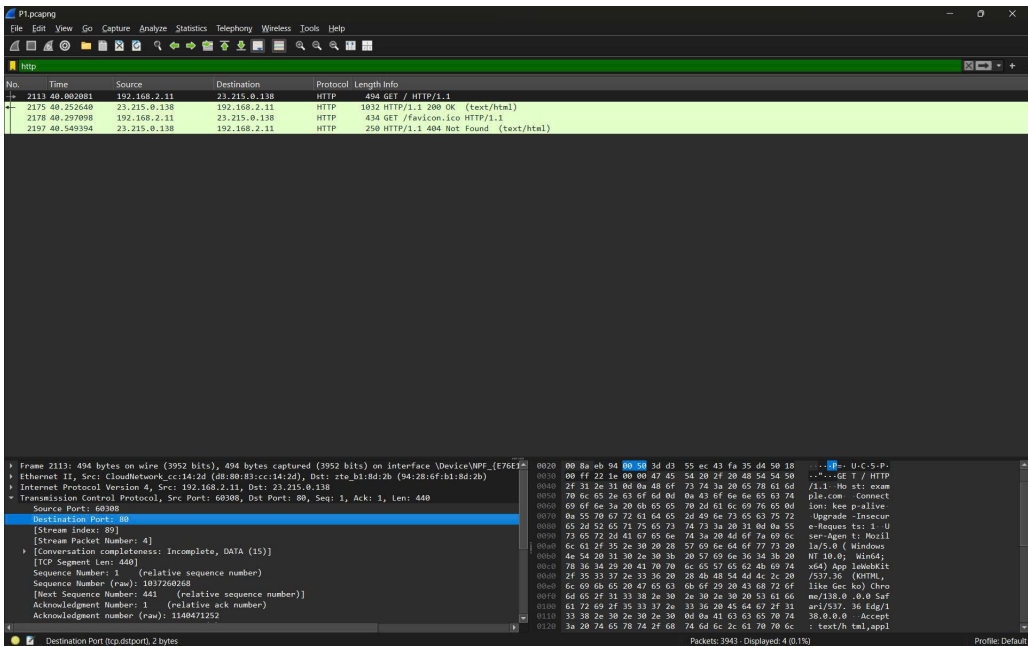
1.2 Phân biệt HTTP & HTTPS

Dựa trên lưu lượng HTTP đã phân tích, có thể khẳng định rằng trang web không sử dụng kết nối an toàn (HTTPS).

Lý do giải thích:

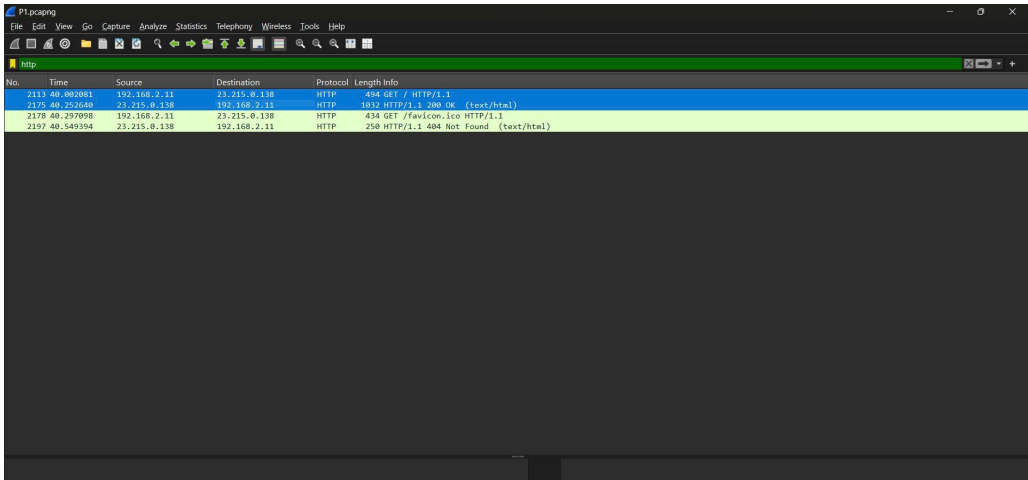
- **Giao thức và Cổng (Protocol and Port):**
 - Các gói tin trong bản chụp Wireshark hiển thị rõ ở cột giao thức sử dụng là HTTP, không phải HTTPS.

- Kết nối được thiết lập trên cổng **TCP 80** mặc định của HTTP, thay vì cổng TCP 443, cổng chuẩn được sử dụng cho HTTPS.



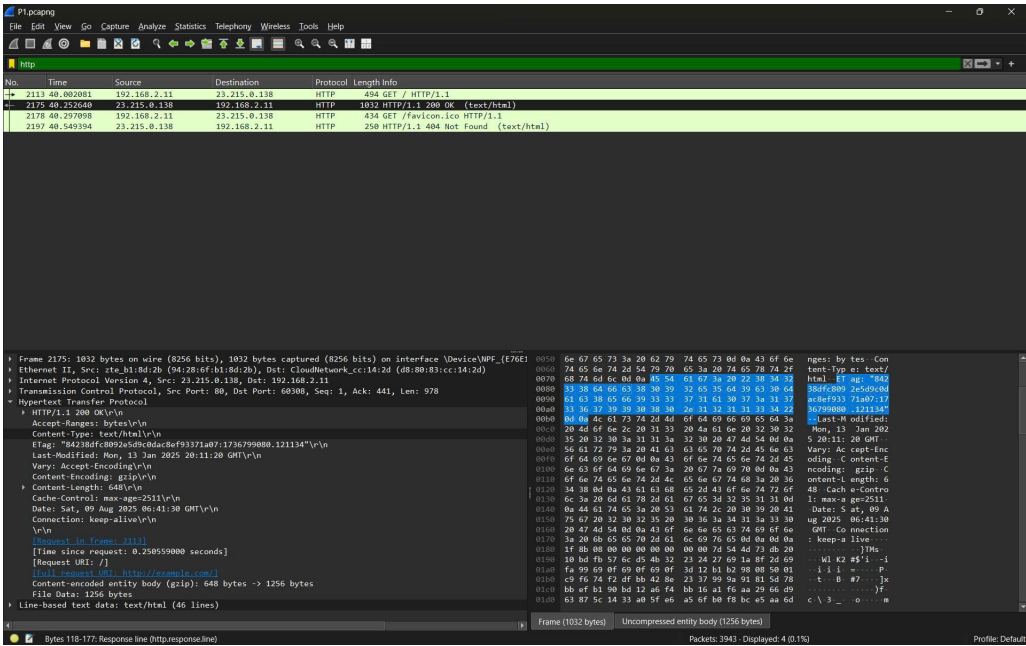
Hình 1.3: Minh chứng HTTP qua Protocol & PortTCP

- **Thiếu quá trình bắt tay TLS (TLS Handshake):**
 - Một kết nối HTTPS yêu cầu một quá trình bắt tay (handshake) bằng giao thức TLS (Transport Layer Security) để thiết lập kênh bảo mật. Quá trình này bao gồm các gói tin như ClientHello và ServerHello.
 - Bản chụp không hiển thị bất kỳ gói tin nào thuộc quá trình bắt tay TLS. Thay vào đó, nó bắt đầu trực tiếp với các yêu cầu (GET) và phản hồi (RESPONE) HTTP.



Hình 1.4: Minh chứng HTTP qua GET & RESPONE

- **Dữ liệu không được mã hóa (Unencrypted Data):**
 - Các tiêu đề và nội dung của yêu cầu và phản hồi HTTP trong bản chụp được hiển thị dưới dạng văn bản thuần túy (plain text).



Hình 1.5: Minh chứng HTTP qua nội dung gói tin

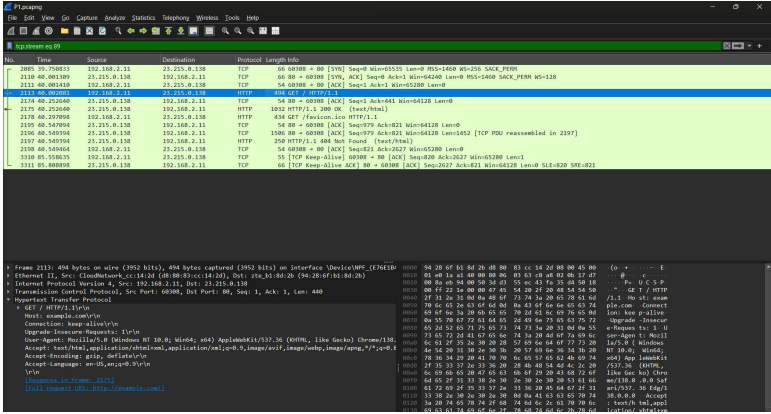
1.3 Kết nối Duy trì & Không Duy trì

Dựa trên phân tích lưu lượng HTTP, có thể xác định được rằng trình duyệt đã sử dụng kết nối HTTP duy trì (persistent connection).

1.3.0.1 Phân tích loại kết nối HTTP

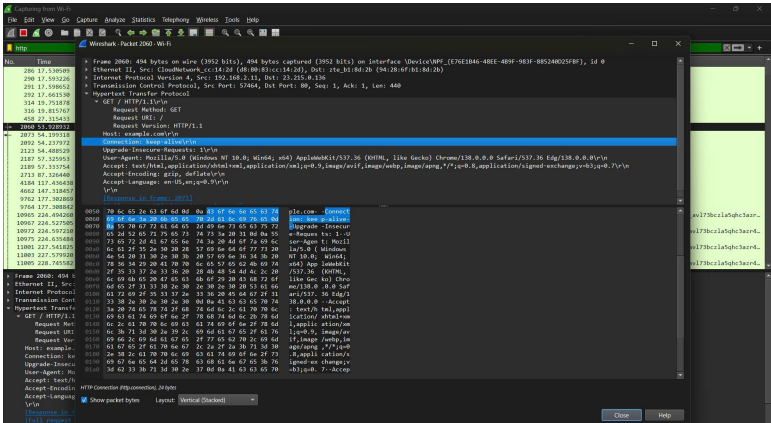
Từ các bằng chứng từ bản chụp Wireshark:

- **Tái sử dụng kết nối TCP:**
 - Các yêu cầu HTTP được gửi đi trên cùng một luồng TCP (tcp.stream == 89).
 - Không có quá trình ngắt kết nối TCP (gói tin FIN) nào giữa hai yêu cầu này, chứng tỏ trình duyệt đã giữ kết nối mở để gửi yêu cầu tiếp theo.



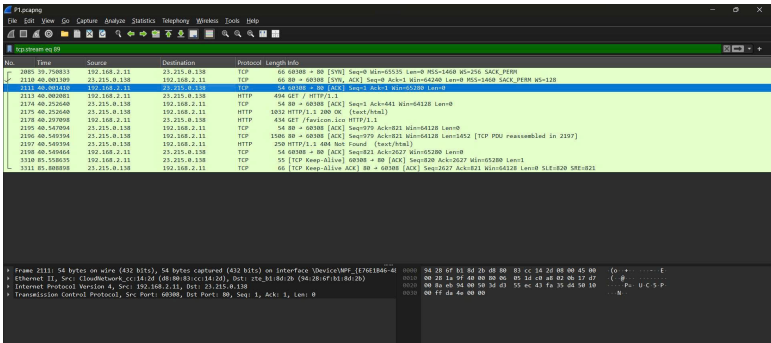
Hình 1.6: Quá trình TCP

- **Tiêu đề HTTP "Connection: keep-alive":**
 - Trong cửa sổ "GET" của Wireshark, tiêu đề Connection: keep-alive sẽ xuất hiện trong cả yêu cầu và phản hồi.



Hình 1.7: Connection: keep alive

- **Không có quá trình thiết lập lại kết nối TCP:**
 - Chỉ có một quá trình bắt tay TCP duy nhất được thực hiện (gói tin 2085 SYN, 2110 SYN, ACK và 2111 ACK) để thiết lập luồng TCP 89, và sau đó nó được sử dụng cho nhiều yêu cầu HTTP mà không cần thiết lập lại.



Hình 1.8: Thực hiện TCP hand-shake

1.3.1 Lợi thế của kết nối HTTP bền vững

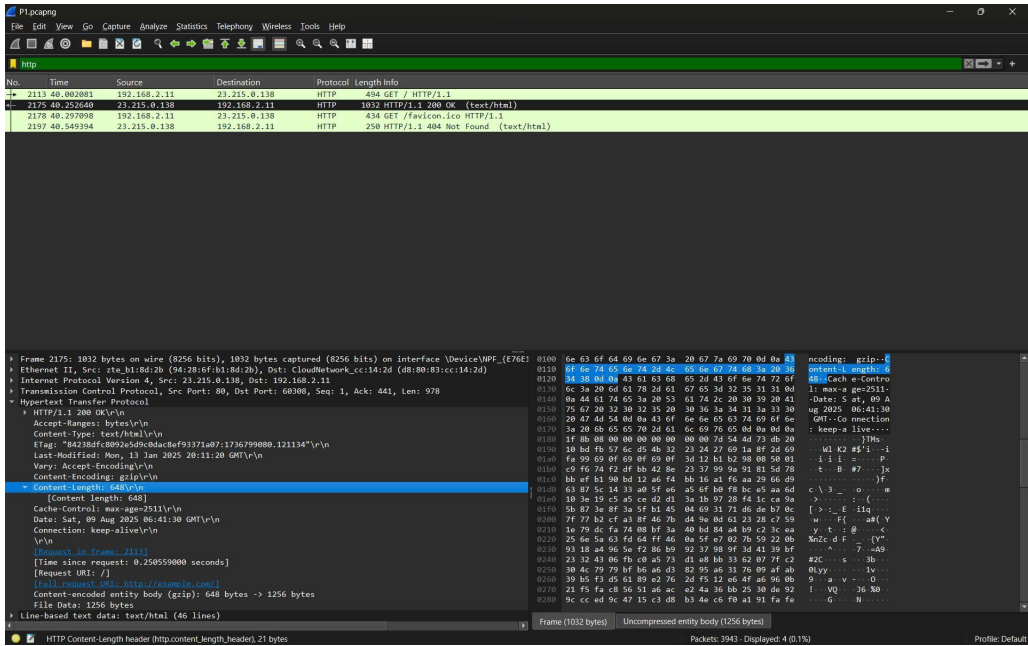
Việc sử dụng kết nối HTTP bền vững mang lại nhiều lợi thế đáng kể cho việc duyệt web, đặc biệt là về hiệu suất:

- **Giảm độ trễ (Lower Latency):** Bằng cách loại bỏ sự cần thiết của quá trình bắt tay TCP (TCP three-way handshake) và bắt tay TLS (TLS handshake nếu dùng HTTPS) cho mỗi đối tượng (như hình ảnh, tệp tin CSS, JavaScript), kết nối bền vững giúp giảm đáng kể thời gian chờ đợi và tăng tốc độ tải trang web.
- **Tăng thông lượng (Higher Throughput):** Khi một kết nối TCP được tái sử dụng, cửa sổ tắc nghẽn (congestion window) của nó đã "đầy lên". Điều này cho phép truyền dữ liệu với tốc độ cao hơn ngay lập tức, tránh được hình phạt của cơ chế "khởi động chậm"(slow-start) mà một kết nối mới phải chịu.
- **Giảm tải tài nguyên máy chủ (Lower Overhead):** Việc tái sử dụng một kết nối giúp giảm số lượng socket cần mở và đóng trên cả máy khách và máy chủ. Điều này giảm bớt gánh nặng xử lý và quản lý tài nguyên, cho phép máy chủ phục vụ nhiều yêu cầu hơn một cách hiệu quả hơn.

1.4 Transfer-Encoding: Chunked

1.4.1 Kiểm tra tiêu đề Transfer-Encoding

Dựa trên phân tích gói tin HTTP/1.1 200 OK, tiêu đề Transfer-Encoding không xuất hiện. Thay vào đó, tiêu đề Content-Length: 648 và Content-Encoding: gzip đã được sử dụng.



Hình 1.9: Transfer-Encoding

Trong giao thức HTTP/1.1, một phản hồi được đóng khung (framed) bằng một trong hai cách: sử dụng Content-Length hoặc Transfer-Encoding: chunked. Sự hiện diện của tiêu đề Content-Length cho biết máy chủ đã biết chính xác kích thước toàn bộ nội dung phản hồi (648 byte) trước khi gửi. Do đó, máy chủ không sử dụng mã hóa chunked.

1.4.2 Lợi ích của Transfer-Encoding: chunked

Máy chủ web sử dụng Transfer-Encoding: chunked khi không thể xác định được tổng kích thước của nội dung phản hồi ngay từ đầu.

Lợi ích chính của phương pháp này là giảm độ trễ và tăng hiệu quả truyền dữ liệu. Thay vì phải chờ đợi cho đến khi toàn bộ nội dung được tạo ra và đo lường kích thước, máy chủ có thể bắt đầu gửi dữ liệu ngay lập tức theo từng "chunk"(khối), mỗi chunk đi kèm với kích thước của nó. Quá trình kết thúc khi máy chủ gửi một chunk có kích thước bằng không. Điều này đặc biệt hữu ích trong các tình huống sau:

- **Tạo phản hồi động:** Khi nội dung được tạo ra từng phần trên máy chủ (ví dụ: các trang web động, luồng dữ liệu).
- **Truyền trực tuyến (streaming):** Khi máy chủ truyền một luồng dữ liệu dài mà không thể biết trước điểm kết thúc (ví dụ: video streaming, báo cáo lớn).

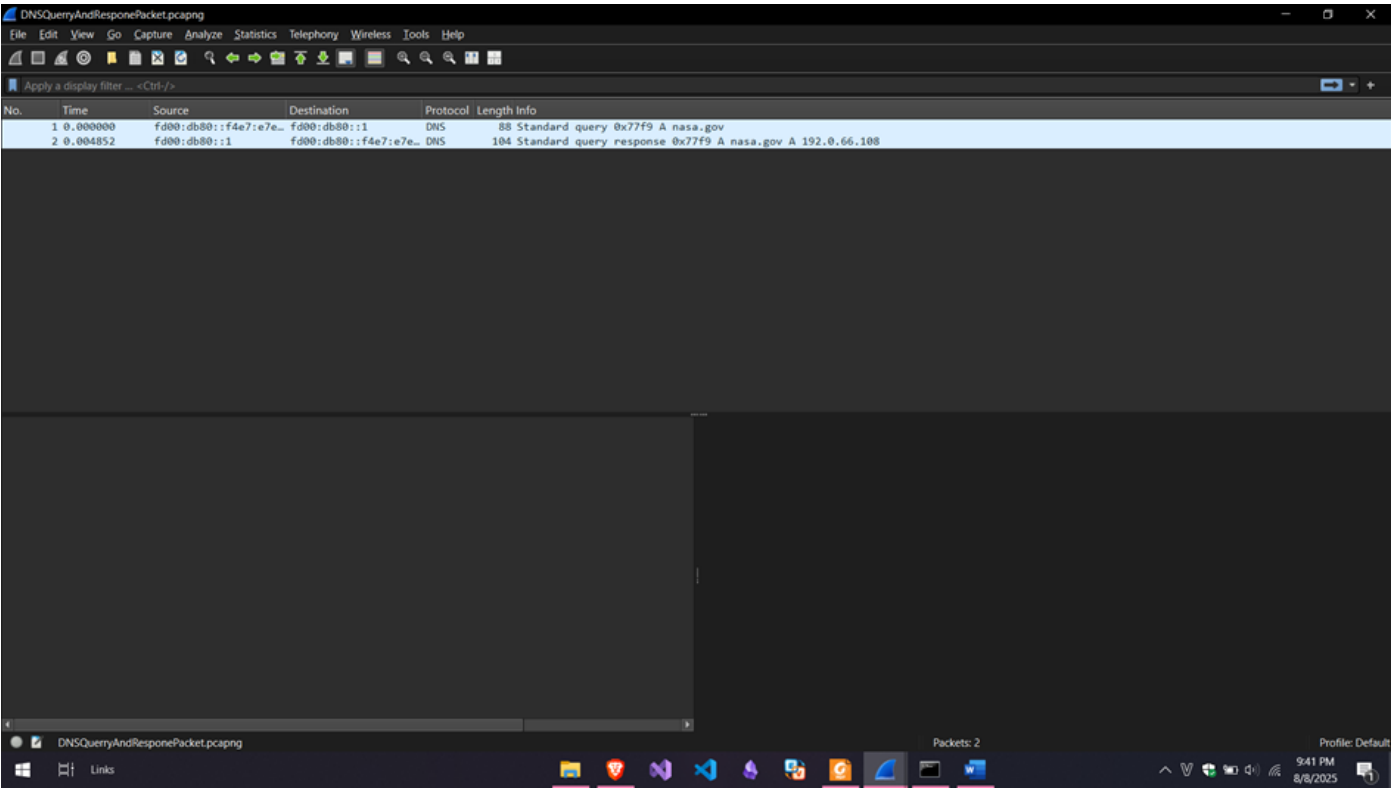
Bằng cách sử dụng mã hóa chunked, máy chủ có thể truyền dữ liệu mà không cần phải đệm (buffer) toàn bộ phản hồi hoặc tính toán Content-Length trước, đồng thời vẫn duy trì được kết nối bền vững (persistent connection).

Phần 2

PHÂN TÍCH LƯU LƯỢNG DNS (IPv4)

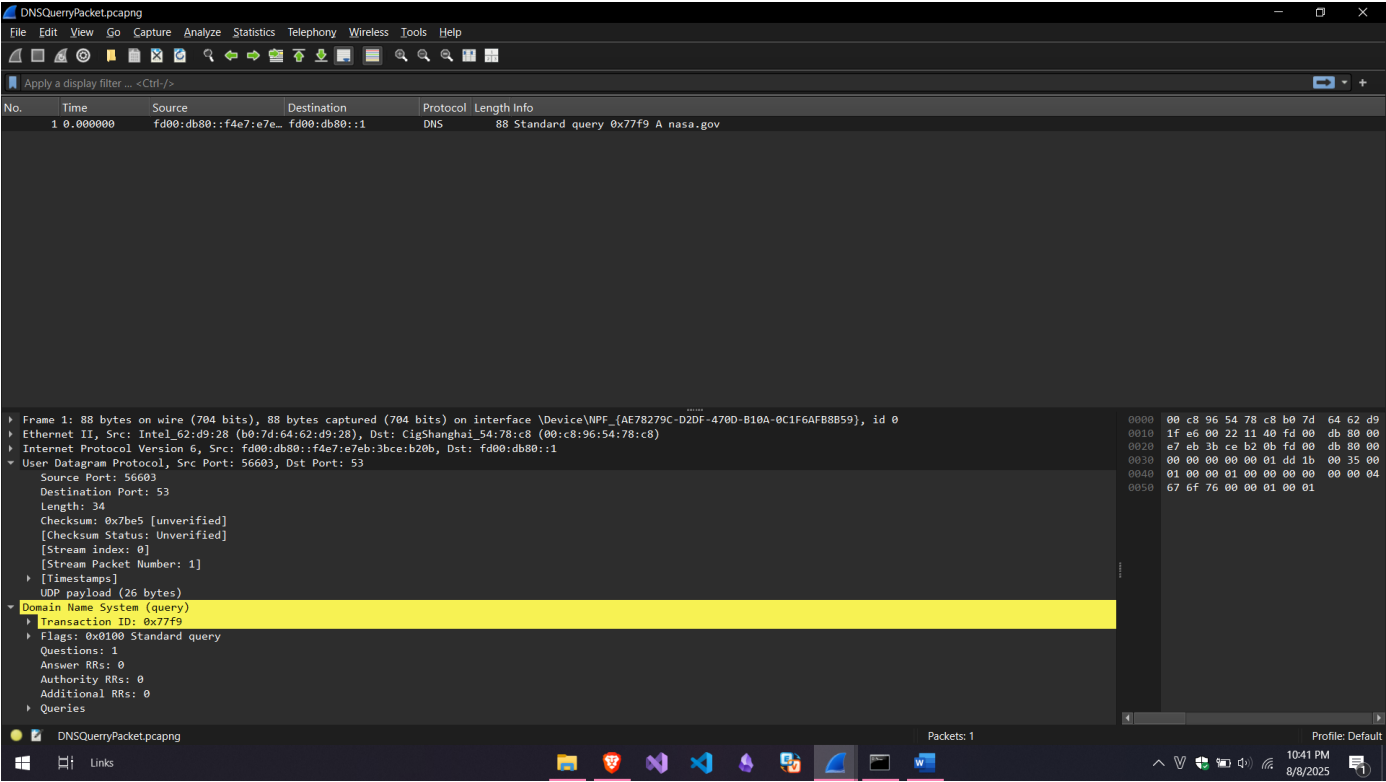
2.1 Tổng quan

Phần này tập trung phân tích các gói tin DNS khi truy vấn địa chỉ IPv4 cho tên miền bất kì (Ví dụ là: `nasa.gov`). Mục tiêu là nhận diện gói tin truy vấn (DNS Query) và phản hồi (DNS Response), xác định giao thức tầng vận chuyển, và lý giải nguyên nhân xuất hiện hoặc không xuất hiện truy vấn cho `example.com` trong quá trình bắt gói.



Hình 2.1: Gói tin DNS Query và Response cho `nasa.gov`

2.2 Gói tin truy vấn DNS: “Loại truy vấn” và “Tên truy vấn” trong gói tin



Hình 2.2: Gói tin DNS Query cho nasa.gov

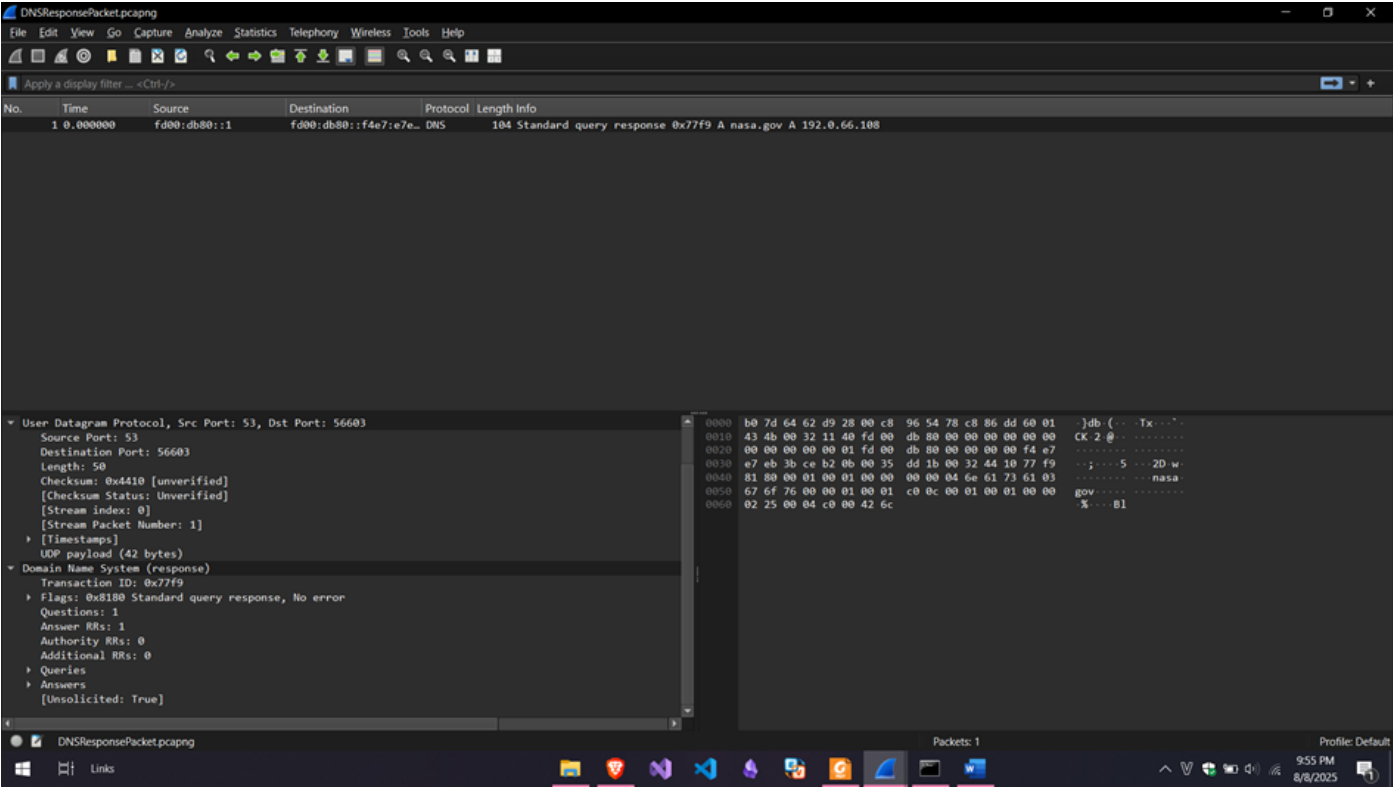
2.2.1 Thông tin gói tin

- **Loại truy vấn:** A (IPv4 address)
- **Tên truy vấn:** nasa.gov
- **Transaction ID:** 0x77f9
- **Giao thức vận chuyển:** UDP
- **Cổng nguồn:** 56603 (ephemeral port)
- **Cổng đích:** 53

2.2.2 Chức năng

Gói tin này được client gửi đến máy chủ DNS để yêu cầu địa chỉ IPv4 của nasa.gov.

2.3 Gói tin phản hồi DNS tương ứng và hiển thị phần “Answer”



Hình 2.3: Gói tin DNS Response cho nasa . gov

2.3.1 Thông tin gói tin

- **Transaction ID:** 0x77f9 (trùng với truy vấn, xác nhận đây là phản hồi)
- **Answer:** nasa.gov A 192.0.66.108
- **Giao thức vận chuyển:** UDP
- **Cổng nguồn:** 53
- **Cổng đích:** 56603

2.3.2 Chức năng

Máy chủ DNS phản hồi thành công và cung cấp địa chỉ IPv4 của nasa.gov để client có thể thiết lập kết nối TCP/UDP.

2.4 Giao thức tầng vận chuyển được DNS sử dụng chủ yếu

2.4.1 Giao thức

UDP (Port 53) được sử dụng mặc định cho các truy vấn DNS thông thường.

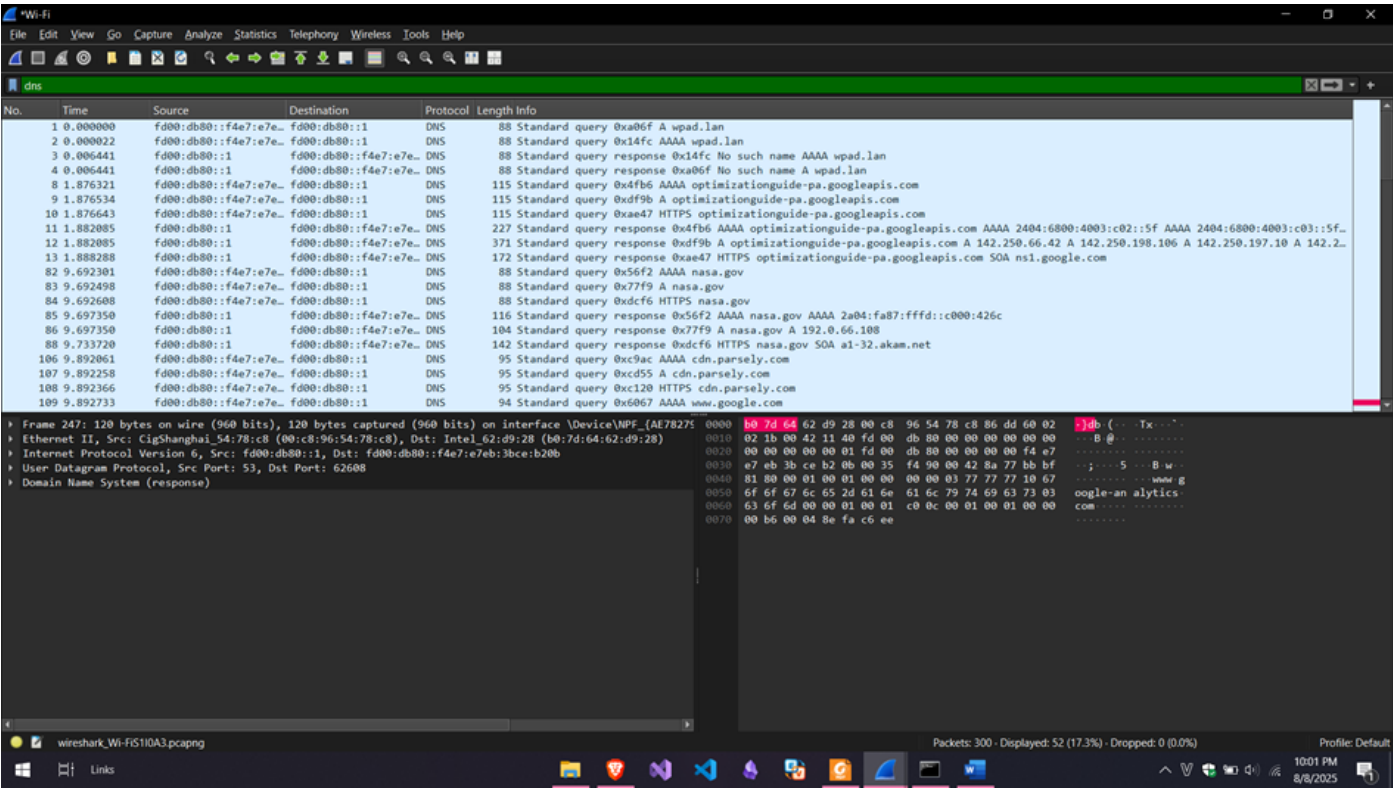
2.4.2 Lý do

- UDP truyền nhanh, ít overhead, phù hợp với truy vấn nhỏ (<512 bytes).

2.4.3 Ngoại lệ khi dùng TCP

- Dữ liệu trả về vượt giới hạn UDP.
- Zone transfer giữa các máy chủ DNS (quá trình đồng bộ dữ liệu vùng (zone file) giữa các máy chủ DNS cần đảm bảo dữ liệu đầy đủ, đúng thứ tự, và không mất mát).
- Sử dụng DNSSEC hoặc cần độ tin cậy cao (TCP được dùng để đảm bảo tất cả dữ liệu đến đúng thứ tự, không bị mất gói, và toàn vẹn).

2.5 Truy vấn DNS cho trang web đã truy cập



Hình 2.4: Toàn bộ gói tin trong capture DNS

2.5.1 Quan sát

Không thấy truy vấn cho `example.com` trong capture.

2.5.2 Các nguyên nhân có thể

- Địa chỉ đã có trong DNS cache của hệ điều hành hoặc router.
- Domain đã được cấu hình trong file `hosts`.

2.5.3 Khi nào sẽ thấy truy vấn `example.com`

- Chưa từng truy cập `example.com` gần đây (cache trống).
- Đã xóa cache DNS trước khi truy cập (`ipconfig /flushdns` trên Windows).
- Domain không tồn tại trong file `hosts` và không bị thay thế bởi proxy hoặc DNS nội bộ.

Phần 3

TÌM HIỂU GIAO THỨC TCP & UDP

3.1 So sánh giao thức TCP và UDP

3.2 So sánh TCP và UDP

- **Kết nối**

- TCP: Hướng kết nối (*connection-oriented*), yêu cầu bắt tay 3 bước trước khi truyền dữ liệu.
- UDP: Không hướng kết nối (*connectionless*), gửi dữ liệu trực tiếp mà không cần thiết lập kết nối.

- **Độ tin cậy**

- TCP: Đảm bảo dữ liệu đầy đủ và đúng thứ tự; có cơ chế xác nhận (ACK) và truyền lại khi mất gói.
- UDP: Không đảm bảo độ tin cậy; không truyền lại khi mất gói.

- **Thứ tự gói tin**

- TCP: Duy trì thứ tự gói tin như khi gửi.
- UDP: Không đảm bảo thứ tự gói tin.

- **Kiểm soát luồng và tắc nghẽn**

- TCP: Có cơ chế kiểm soát luồng và tắc nghẽn để điều chỉnh tốc độ gửi dữ liệu.
- UDP: Không có cơ chế này, tốc độ gửi phụ thuộc vào ứng dụng.

- **Kích thước header**

- TCP: Tối thiểu 20 byte, tối đa 60 byte (nhiều trường điều khiển).
- UDP: Cố định 8 byte (gọn nhẹ, ít overhead).

- **Tốc độ và độ trễ**
 - TCP: Chậm hơn do nhiều cơ chế đảm bảo dữ liệu, độ trễ cao hơn UDP.
 - UDP: Nhanh hơn, độ trễ thấp, phù hợp ứng dụng thời gian thực.
- **Ứng dụng tiêu biểu**
 - TCP: HTTP/HTTPS, FTP, Email (SMTP, IMAP, POP3), truyền file yêu cầu toàn vẹn dữ liệu.
 - UDP: VoIP, gọi video, streaming, game online, DNS, ứng dụng cần tốc độ cao và chấp nhận mất gói.

Bảng 3.1: So sánh TCP và UDP

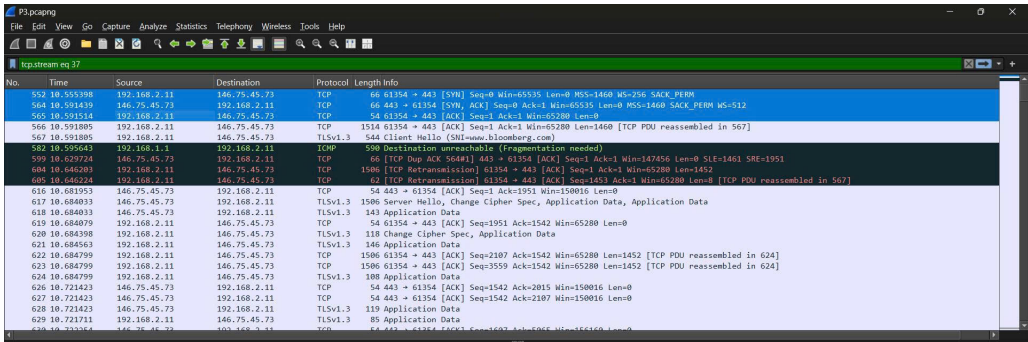
Tiêu chí	TCP	UDP
Kết nối	Hướng kết nối, cần bắt tay 3 bước trước khi truyền dữ liệu.	Không hướng kết nối, gửi dữ liệu trực tiếp.
Độ tin cậy	Đảm bảo dữ liệu đầy đủ, đúng thứ tự, có ACK và truyền lại khi mất gói.	Không đảm bảo độ tin cậy, không truyền lại khi mất gói.
Thứ tự gói tin	Đảm bảo đúng thứ tự khi nhận.	Không đảm bảo thứ tự gói tin.
Kiểm soát luồng/tắc nghẽn	Có kiểm soát luồng và tắc nghẽn.	Không có, tốc độ phụ thuộc ứng dụng.
Kích thước header	20–60 byte.	8 byte.
Tốc độ và độ trễ	Chậm hơn, độ trễ cao hơn do nhiều cơ chế đảm bảo dữ liệu.	Nhanh hơn, độ trễ thấp, phù hợp thời gian thực.
Ứng dụng	HTTP/HTTPS, FTP, Email.	VoIP, gọi video, streaming, game online, DNS.

3.3 Lưu lượng TCP

3.3.1 Phân tích quá trình bắt tay TCP (TCP Handshake)

Quá trình bắt tay ba bước của TCP (three-way handshake) được hiển thị rõ ràng qua ba

gói tin đầu tiên, thiết lập một kết nối an toàn (HTTPS) giữa máy khách (192.168.2.11) và máy chủ (146.75.45.73).

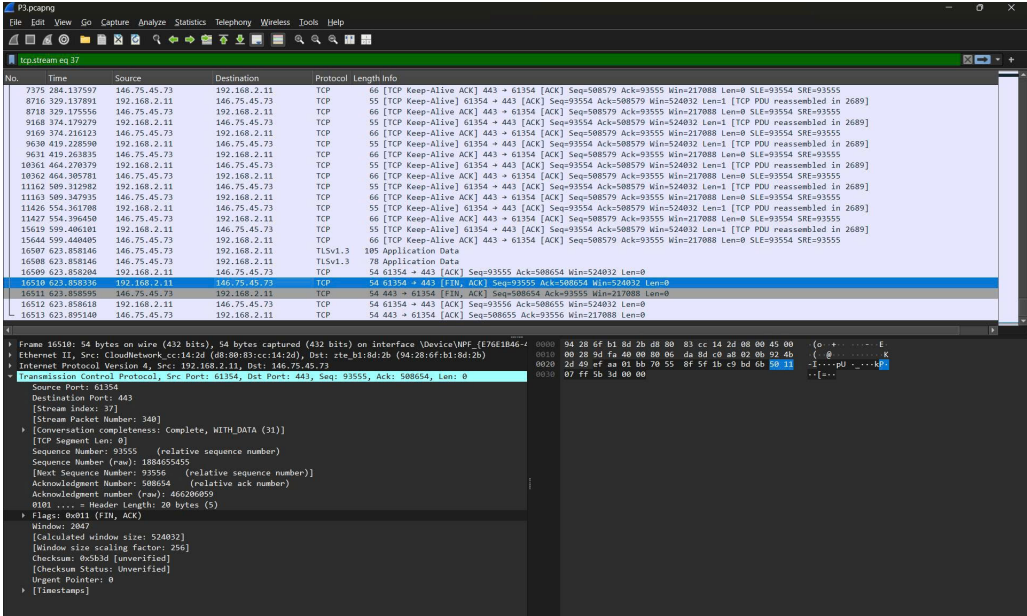


Hình 3.1: TCP HandShake

- Gói tin 552 ([SYN]):
 - Nguồn: 192.168.2.11:61354
 - Đích: 146.75.45.73:443
 - Mục đích: Gói tin này được máy khách gửi để **khởi đầu kết nối**. Mang cờ SYN (Synchronize) để đồng bộ hóa số thứ tự ban đầu và đề xuất các tùy chọn TCP như MSS (Maximum Segment Size).
- Gói tin 566 ([SYN, ACK]):
 - Nguồn: 146.75.45.73:443
 - Đích: 192.168.2.11:61354
 - Mục đích: Đây là phản hồi của máy chủ. Nó mang cờ SYN của máy chủ và ACK (Acknowledgement) để **xác nhận** đã nhận được yêu cầu SYN của máy khách.
- Gói tin 567 ([ACK]):
 - Nguồn: 192.168.2.11:61354
 - Đích: 146.75.45.73:443
 - Mục đích: Gói tin này hoàn tất quá trình bắt tay bằng cách mang cờ ACK để **xác nhận** SYN của máy chủ. Từ gói tin này trở đi, kết nối TCP đã được thiết lập và có thể bắt đầu truyền dữ liệu ứng dụng. Trong trường hợp này, gói tin ACK cũng mang theo tải trọng đầu tiên của lớp ứng dụng, đó là thông điệp TLS ClientHello.

3.3.2 Phân tích quá trình ngắt kết nối TCP (TCP Teardown)

Quá trình ngắt kết nối TCP được gọi là "đóng kết nối có trật tự"(graceful close) và được thực hiện bằng cách trao đổi cờ FIN (Finish). Dữ liệu được cung cấp cho thấy quá trình này đã bắt đầu.



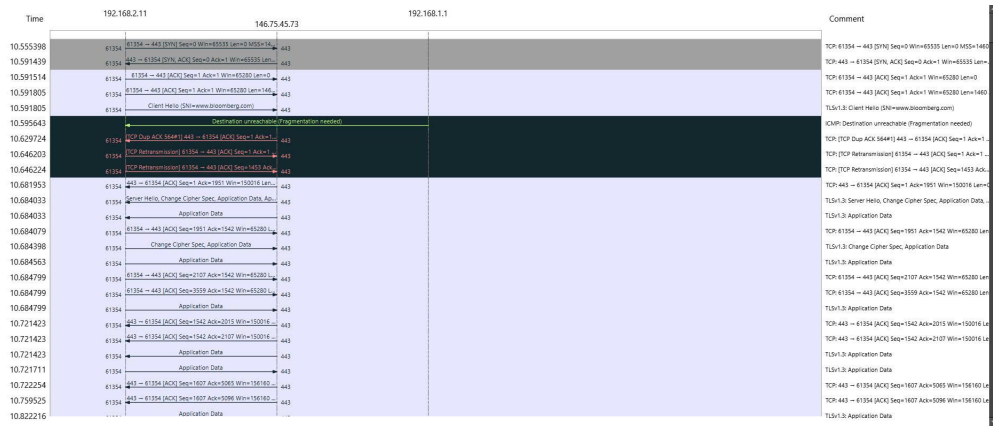
Hình 3.2: FIN, ACK

- Gói tin 16510 ([FIN, ACK]):
 - Nguồn: 192.168.2.11:61354
 - Đích: 146.75.45.73:443
 - Mục đích: Máy khách gửi gói tin này để báo hiệu rằng nó đã hoàn thành việc gửi dữ liệu. Tuy nhiên, kết nối vẫn mở để máy khách có thể tiếp tục nhận dữ liệu từ máy chủ.
- Gói tin 16511 ([ACK]):
 - Nguồn: 146.75.45.73:443
 - Đích: 192.168.2.11:61354
 - Mục đích: Máy chủ gửi gói tin này để xác nhận đã nhận được cờ FIN của máy khách.

Để hoàn tất quá trình đóng kết nối, máy chủ cũng sẽ gửi một gói tin [FIN, ACK] của riêng nó, và máy khách sẽ phản hồi bằng một gói tin [ACK] cuối cùng. Nếu kết nối bị ngắt đột ngột (ví dụ: do lỗi), một gói tin [RST] (Reset) sẽ được sử dụng thay thế.

3.3.3 Các cổng được sử dụng

- **Cổng nguồn (61354):** Đây là một cổng tạm thời (ephemeral port) được hệ điều hành của máy khách chọn ngẫu nhiên để khởi tạo kết nối.
- **Cổng đích (443):** Đây là cổng chuẩn cho giao thức **HTTPS**, cho thấy kết nối này được mã hóa bằng TLS/SSL. Nếu là kết nối HTTP không an toàn, cổng đích sẽ là 80.



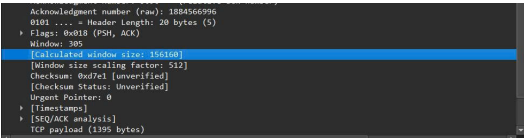
Hình 3.3: Cổng nguồn - Cổng đích đến

3.3.4 Giải thích Window size value và Calculated window size

Đây là hai giá trị liên quan đến cơ chế kiểm soát luồng (flow control) của TCP, quyết định lượng dữ liệu mà một bên có thể gửi đi trước khi phải nhận được xác nhận (ACK).

- **Window size value:** Đây là giá trị thô 16-bit trong tiêu đề TCP, được bên gửi gói tin quảng cáo. Nó đại diện cho số byte dữ liệu bổ sung mà bên nhận sẵn sàng chấp nhận sau số thứ tự (sequence number) được xác nhận gần nhất. Giá trị này có giới hạn tối đa là 65,535 byte.
- **Calculated window size:** Đây là kích thước cửa sổ hiệu quả (effective window size) sau khi áp dụng Hệ số chia tỷ lệ cửa sổ (Window Scale) đã được thương lượng trong quá trình bắt tay (handshake). Wireshark tính toán giá trị này bằng cách nhân giá trị thô với hệ số chia tỷ lệ.

Trong giao thức TCP này, cờ mà các gói tin mang theo biểu thị các giá trị như sau:

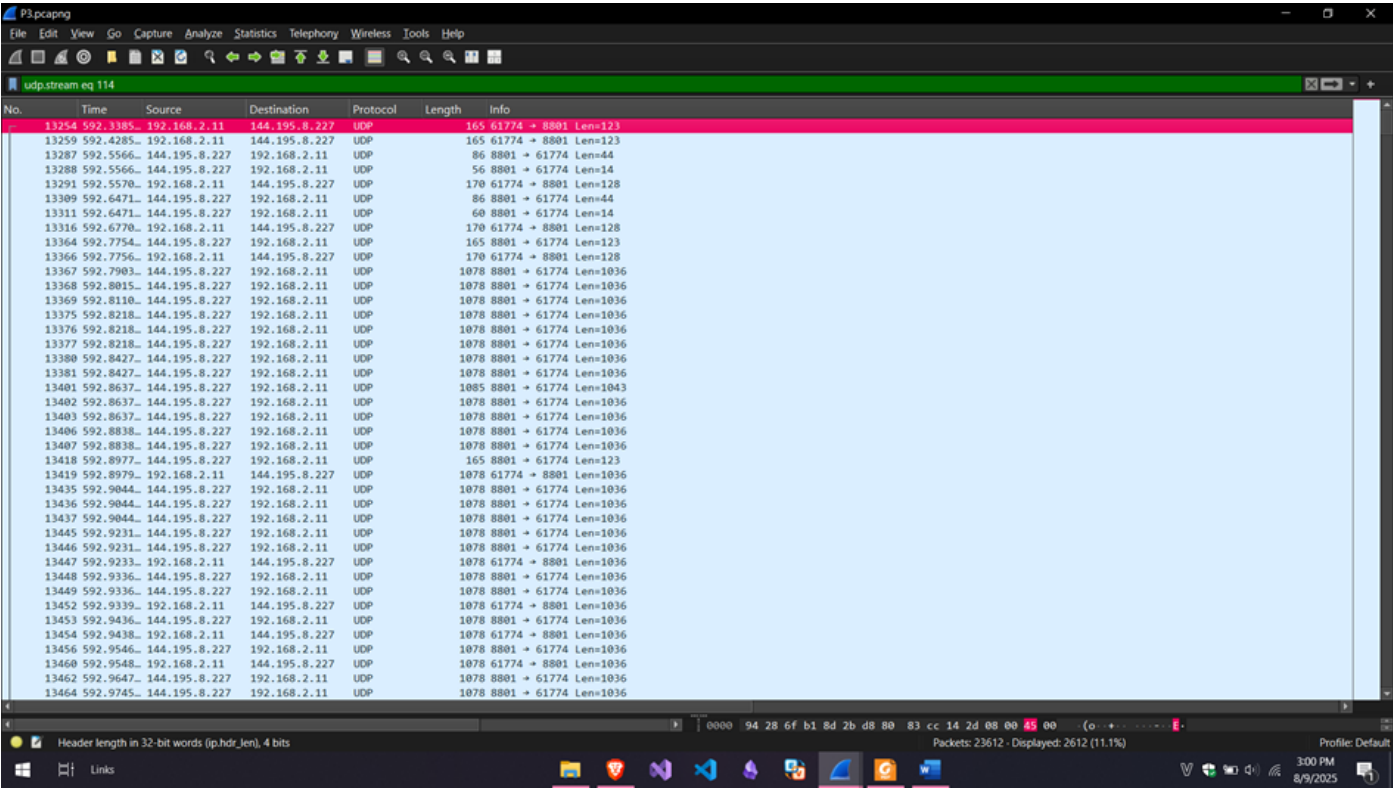


Hình 3.4: Window Size

- **Flags: PSH, ACK:** Gói tin mang dữ liệu này có cả hai cờ PSH (Push) và ACK (Acknowledgement). PSH ra hiệu cho bên nhận rằng dữ liệu này nên được chuyển lên lớp ứng dụng ngay lập tức, thay vì chờ đợi bộ đệm đầy. ACK xác nhận rằng bên gửi gói tin đã nhận được tất cả các byte dữ liệu tính đến số thứ tự ACK được hiển thị.
- **Window (255 raw):** Đây là giá trị thô 16-bit của trường cửa sổ nhận (receive window) trong tiêu đề TCP. Nó cho biết bên nhận sẵn sàng tiếp nhận thêm 255 byte dữ liệu nữa.
- **Window size scaling factor (256):** Hệ số chia tỷ lệ cửa sổ, đã được thương lượng trong quá trình bắt tay TCP. Hệ số 256 tương đương với dịch bit 8 (`shift=8`).
- **Calculated window size (65280):** Wireshark đã tính toán giá trị này bằng cách nhân giá trị thô (255) với hệ số chia tỷ lệ (256), cho ra kết quả 65,280 byte. Đây là kích thước cửa sổ nhận hiệu quả, cho biết bên đối diện có thể gửi thêm tối đa 65,280 byte dữ liệu mà không cần chờ ACK.

3.4 Lưu lượng UDP

3.4.1 Gói tin UDP có cơ chế bắt tay tương tự TCP không?

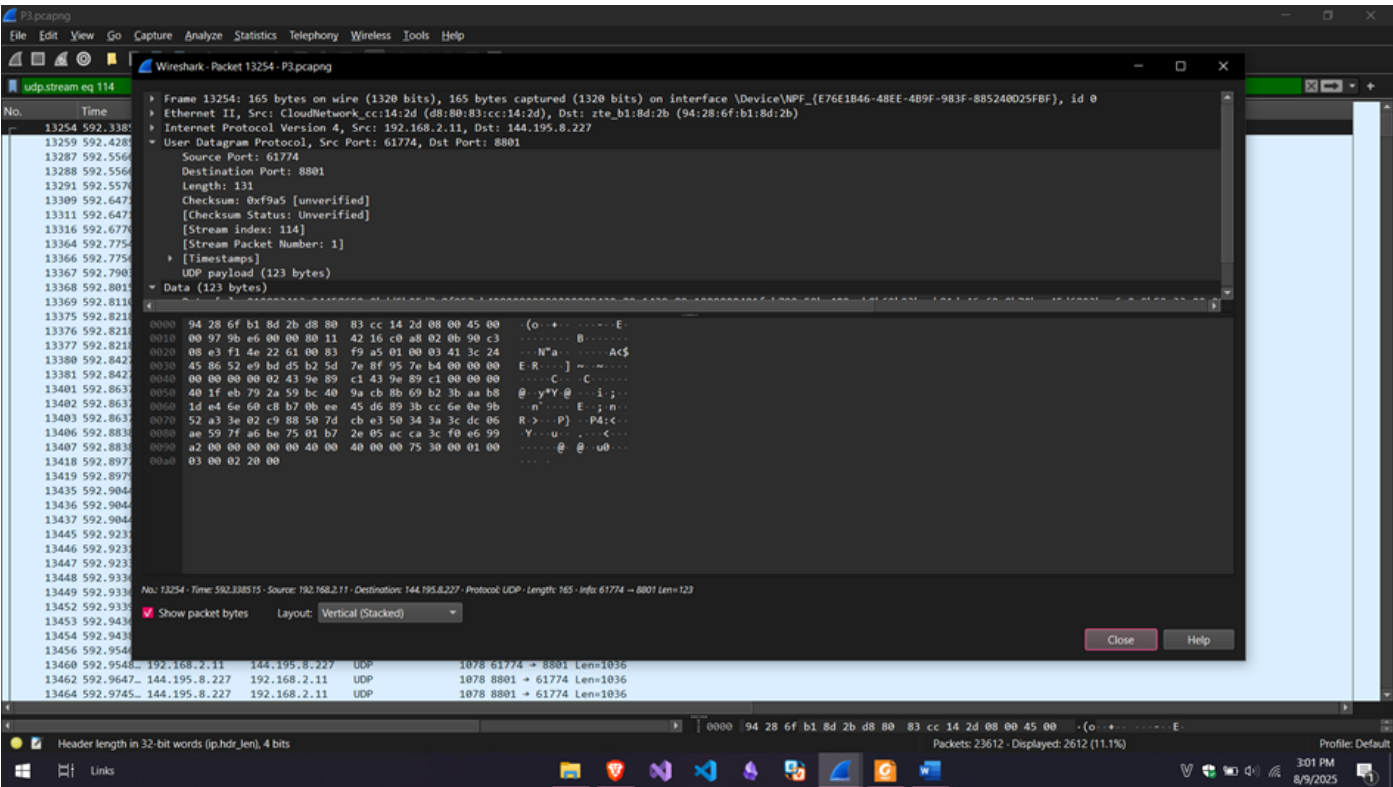


Hình 3.5: Toàn bộ gói tin trong capture UDP

Không giống như TCP, giao thức UDP không có cơ chế bắt tay ba bước (SYN, SYN-ACK, ACK). UDP gửi dữ liệu trực tiếp đến đích mà không cần thiết lập kết nối trước. Việc bỏ qua các bước này giúp:

- Giảm độ trễ (*low latency*), rất quan trọng trong truyền dữ liệu thời gian thực.
- Phù hợp cho các ứng dụng thời gian thực như VoIP, gọi video, hoặc game trực tuyến, nơi việc mất một vài gói tin không quan trọng bằng tốc độ truyền.

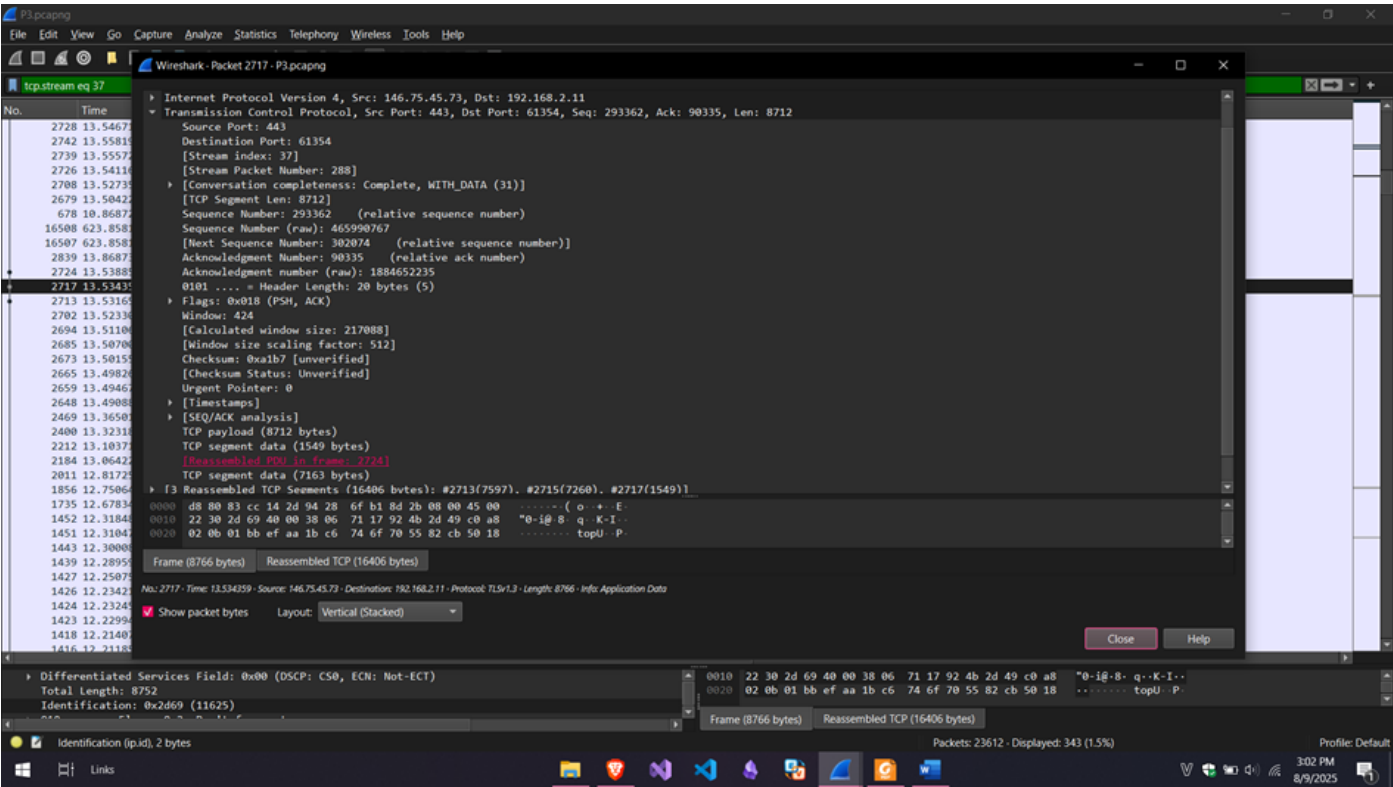
3.4.2 Các thông tin trong header UDP và so sánh với header TCP



Hình 3.6: Header của gói tin UDP

Header UDP (8 byte) bao gồm:

- Source Port (2 byte) – Cổng nguồn.
- Destination Port (2 byte) – Cổng đích.
- Length (2 byte) – Độ dài toàn bộ gói UDP (header + dữ liệu).
- Checksum (2 byte) – Kiểm tra lỗi gói tin.



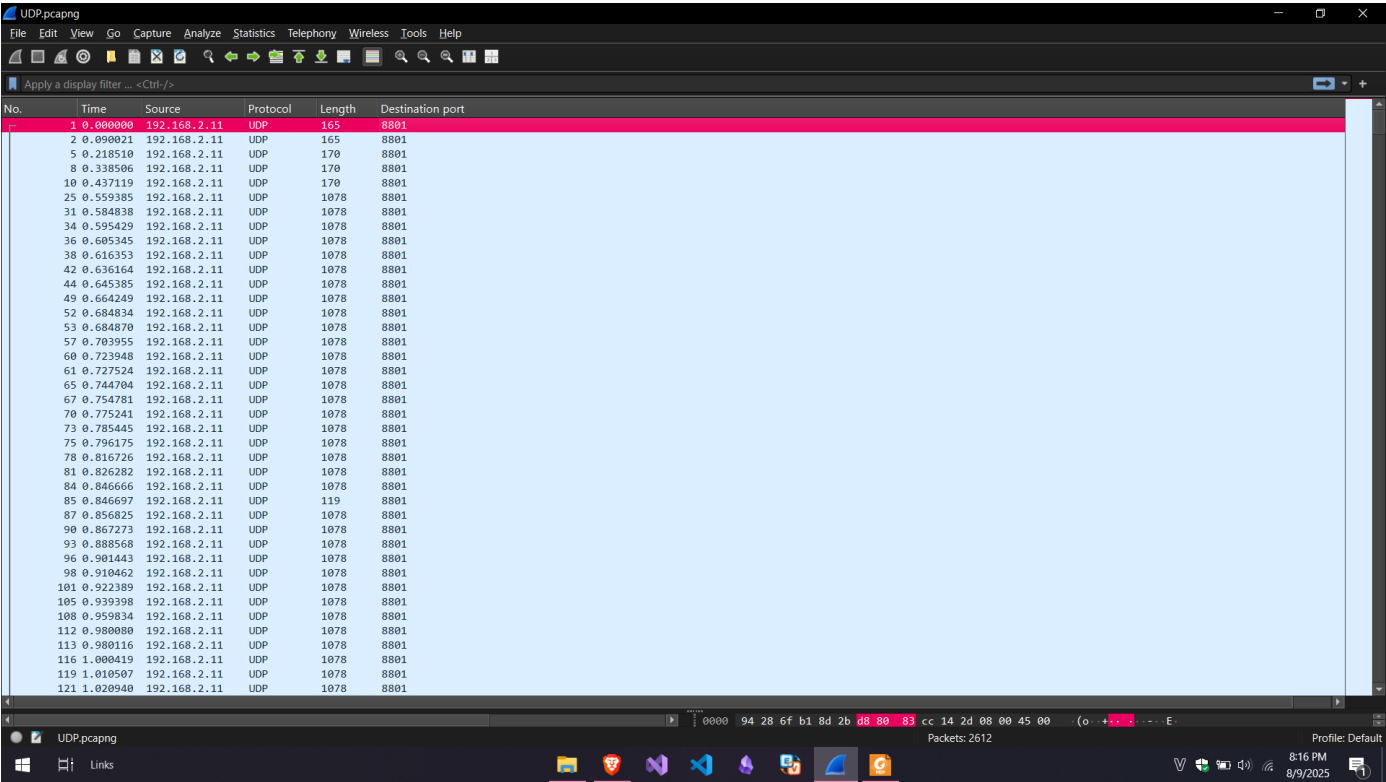
Hình 3.7: Header của gói tin TCP

So sánh với UDP, Header TCP gồm nhiều trường hơn (tối thiểu 20 byte, tối đa 60 byte):

- Source Port (2 byte) – Cổng gửi dữ liệu (ví dụ: 443 cho HTTPS).
- Destination Port (2 byte) – Cổng nhận dữ liệu (ví dụ: cổng tạm 61354 của client).
- Sequence Number (4 byte) – Vị trí byte đầu tiên trong segment.
- Acknowledgment Number (4 byte) – Byte tiếp theo mong muốn nhận nếu chờ ACK bật.
- Data Offset (4 bit) – Độ dài TCP header.
- Reserved (3 bit) – Luôn bằng 0.
- Flags (9 bit) – Bao gồm URG, ACK, PSH, RST, SYN, FIN (và NS, CWR, ECE cho mở rộng).
- Window Size Value (2 byte) – Số byte mà bên nhận sẵn sàng chấp nhận.
- Checksum (2 byte) – Kiểm tra lỗi.

- Urgent Pointer (2 byte) – Chỉ dữ liệu khẩn nếu URG=1.
- Options (0–40 byte) – Các tùy chọn như MSS, Window Scale, Timestamp.
- Padding – Thêm bit 0 để đủ bội số 32 bit.

3.4.3 Các loại ứng dụng và dịch vụ sử dụng UDP



Hình 3.8: Các Destination Port khi bắt gói tin UDP

Dải cổng theo chuẩn IANA được phân chia như sau:

- 0–1023: Well-known ports (cổng chuẩn), ví dụ: 80 cho HTTP, 443 cho HTTPS, 53 cho DNS...
- 1024–49151: Registered ports (cổng tùy chỉnh), được đăng ký cho các ứng dụng nhất định.
- 49152–65535: Dynamic/Ephemeral ports (cổng tạm thời), do hệ điều hành chọn ngẫu nhiên cho kết nối.

Dữ liệu bắt được cho thấy lưu lượng UDP có Source Port 61774, Destination Port 8801. Cổng 8801 thường được sử dụng bởi các ứng dụng tùy chỉnh, phổ biến trong game trực tuyến hoặc ứng dụng truyền thông video và gọi điện thoại thời gian thực (riêng trong trường hợp này, nhóm đã lựa chọn ứng dụng Zoom để kiểm tra).

Phần 4

GIẢI QUYẾT VẤN ĐỀ GIẢ ĐỊNH

4.1 TẠO MÔI TRƯỜNG GIẢ ĐỊNH

- Truy cập đường dẫn `C:\Windows\System32\drivers\etc`
- Mở file hosts, dán dòng `10.255.255.1 www.example.com` vào trong file hosts và lưu lại. Việc này sẽ map domain `www.example.com` đến một địa chỉ IP không tồn tại.
- Mở Command Prompt, chạy: `ipconfig /flushdns` để xóa cache DNS.

4.2 CHẨN ĐOÁN NGUYÊN NHÂN GÂY LỖI

Để tiến hành xác định nguyên nhân gây lỗi, ta cần làm các bước sau:

- Bắt đầu một Capture mới ở interface đang dùng (Ethernet/Wifi) trong Wireshark.
- Tiến hành truy cập vào website bị chặn (`www.example.com`, được map tới địa chỉ `10.255.255.1` trong file host) bằng trình duyệt web.
- Sau khi trình duyệt web báo lỗi không truy cập được thì ngưng bản Capture và tiến hành phân tích
- Lần lượt áp dụng các display filter để phân tích:
 - `dns` để xem có truy vấn DNS hay không.
 - `icmp` để xem có gói ICMP báo unreachable không.
 - `tcp.flags.syn == 1 && tcp.flags.ack == 0` để tìm gói SYN khởi tạo kết nối.
 - `tcp.analysis.retransmission` để xem gói bị gửi lại.
 - `tcp.flags.reset == 1` để tìm gói TCP Reset (RST).

- `tcp.flags.syn == 1 && tcp.flags.ack == 1` để tìm SYN-ACK đối chiếu với kết nối thành công.

4.3 NHỮNG GIAO THỨC VÀ GÓI CẦN XEM XÉT

- DNS: kiểm tra truy vấn phân giải tên miền.
- TCP: bắt tay 3 bước (`SYN` → `SYN-ACK` → `ACK`), các gói retransmission, reset.
- ICMP: tìm gói Destination Unreachable hoặc Echo Reply/Request.
- TLS/HTTP: kiểm tra xem sau khi TCP thành công có handshake TLS hoặc HTTP GET không.

4.4 XÁC ĐỊNH CỤ THỂ NGUYÊN NHÂN GÂY LỖI

4.4.1 DNS

- Mong chờ: Khi trình duyệt truy cập `www.example.com` bạn sẽ thấy Standard query cho `www.example.com` và Standard query response trả về 1 hoặc nhiều IP.
- Thực tế: Có query/responses nhưng cho `skydrive.wns.windows.com`. Không có query/response cho `www.example.com`.
- Ý nghĩa: Trình duyệt không thực hiện lookup DNS cho `www.example.com` - Điều này thường xảy ra nếu tên đã được resolve cục bộ (ví dụ file hosts).



No.	Time	Source	Destination	Protocol	Length	Info
59	3.419674	192.168.0.237	1.1.1.1	DNS	84	Standard query 0x4e3b A skydrive.wns.windows.com
61	3.432305	1.1.1.1	192.168.0.237	DNS	164	Standard query response 0x4e3b A skydrive.wns.windows.com CNAME client.wns.windows.com CNAME wns.notify.trafficmanager.net A 4.213.25.241

Hình 4.1: Filter DNS

4.4.2 TCP

Khi dùng filter `tcp.flags.syn == 1 && tcp.flags.ack == 0` (SYN từ client):

- Mong chờ: Thấy SYN từ client tới IP đích, và ngay sau đó sẽ có SYN,ACK từ server. Khi bắt tay thành công thì sẽ có ACK của client và mẫu TLS/HTTP tiếp theo.

- Thực tế: Nhiều dòng gắn nhãn [TCP Retransmission] đi từ source 192.168.0.237 gửi SYN đi nhưng không có SYN-ACK phản hồi cho các lần này. Một vài SYN nhắm tới 107.167.100.100, và có các SYN nhắm tới 10.255.255.1
- Ý nghĩa: Client đã gửi SYN nhưng không nhận được phản hồi (no SYN-ACK) → TCP 3-way handshake không hoàn tất, có thể khả năng do đích đến không tồn tại (thực tế) hoặc gói bị drop trên đường (bị block).

tcp.flags.syn == 1 && tcp.flags.ack == 0						
No.	Time	Source	Destination	Protocol	Length	Info
32	2.093619	192.168.0.237	107.167.100.100	TCP	66	60055 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
33	2.353308	192.168.0.237	107.167.100.100	TCP	66	60057 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
35	2.573241	192.168.0.237	107.167.100.100	TCP	66	60059 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
52	2.821116	192.168.0.237	107.167.100.100	TCP	66	60061 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
53	2.964624	192.168.0.237	107.167.100.100	TCP	66	60063 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
55	3.106267	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60055 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
58	3.353822	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60057 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
60	3.420954	192.168.0.237	107.167.100.100	TCP	66	60065 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
62	3.432954	192.168.0.237	4.213.25.241	TCP	66	60067 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
68	3.583250	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60059 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
87	3.832220	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60061 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
93	3.971115	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60063 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
152	4.432471	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60065 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
155	4.722405	192.168.0.237	10.255.255.1	TCP	66	60074 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
163	4.972959	192.168.0.237	10.255.255.1	TCP	66	60078 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
165	5.109566	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60055 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
170	5.247623	192.168.0.237	10.255.255.1	TCP	66	[TCP Port numbers reused] 60074 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
172	5.354771	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60057 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
175	5.480149	192.168.0.237	10.255.255.1	TCP	66	[TCP Port numbers reused] 60078 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
178	5.595765	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60059 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
180	5.764938	192.168.0.237	10.255.255.1	TCP	66	[TCP Port numbers reused] 60074 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
183	5.844337	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60061 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
185	5.983857	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60063 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
186	5.990137	192.168.0.237	10.255.255.1	TCP	66	[TCP Port numbers reused] 60078 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
192	6.277540	192.168.0.237	10.255.255.1	TCP	66	[TCP Port numbers reused] 60074 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
194	6.434918	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60065 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
195	6.512053	192.168.0.237	10.255.255.1	TCP	66	[TCP Port numbers reused] 60078 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
203	6.789961	192.168.0.237	10.255.255.1	TCP	66	[TCP Port numbers reused] 60074 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM

Frame 152: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface DeviceVMP (f566f88-ef58-4462-9646-7989292ffdb), id 0

Ethernet II, Src: ASUSTekCOMPU 22:fa:62 (04:42:1a:22:fa:62), Dst: TPLink 38:bc:44 (14:eb:b6:38:bc:44)

Internet Protocol Version 4, Src: 192.168.0.237, Dst: 107.167.100.100

Transmission Control Protocol, Src Port: 60066, Dst Port: 443, Seq: 0, Len: 0

0000 14 eb b6 38 bc 44 04 02 1a 22 fa 62 08 00 45 00 ..8 D B .. b E
0010 00 34 b9 53 40 00 08 06 00 00 c0 a8 00 ed 6b a7 4 5gk
0020 64 64 ea a2 01 bb e8 32 8a e5 00 00 00 00 80 02 dd.....2
0030 ff ff 91 c7 00 00 02 04 05 b4 01 03 03 08 01 01
0040 04 02

Hình 4.2: SYN From Client

Khi dùng filter `tcp.flags.syn == 1 && tcp.flags.ack == 1`:

- Mong chờ: Nếu server trả lời, mỗi SYN sẽ có SYN-ACK kèm theo (source = server IP).
- Thực tế: Chỉ có 1 dòng kết nối thành công tới server khác.
- Ý nghĩa: Không có SYN-ACK phản hồi.

tcp.flags.syn == 1 && tcp.flags.ack == 1						
No.	Time	Source	Destination	Protocol	Length	Info
63	3.528707	4.213.25.241	192.168.0.237	TCP	66	443 → 60067 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1440 WS=1 SACK_PERM

Hình 4.3: SYN và ACK

Khi dùng filter `tcp.analysis.retransmission`:

- Mong chờ: Thường có rất ít các retransmission, nếu nhiều thì có vấn đề.

- Thực tế: Rất nhiều retransmission tới cùng các port (ví dụ client port 60055/60057 → 443). Wireshark đánh dấu [TCP Retransmission].
- Ý nghĩa: Retransmission thể hiện client đã không nhận ACK/SYN-ACK → TCP phải gửi lại. Điều này củng cố rằng không có phản hồi từ đích.

tcp.analysis.retransmission									
No.	Time	Source	Destination	Protocol	Length	Info			
55	3.106267	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60055 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
58	3.353822	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60057 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
68	3.583250	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60059 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
87	3.832228	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60061 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
93	3.971115	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60063 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
152	4.433471	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60066 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
165	5.109566	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60055 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
172	5.354771	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60057 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
178	5.595765	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60059 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
183	5.844337	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60061 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
185	5.983857	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60063 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
194	6.434918	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60066 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
253	9.117702	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60055 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
267	9.366006	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60057 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
274	9.597455	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60059 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
283	9.852840	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60061 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
286	9.991275	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60063 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			
293	10.439738	192.168.0.237	107.167.100.100	TCP	66	[TCP Retransmission] 60066 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM			

Hình 4.4: Retransmission

Khi dùng filter `tls.handshake` và `http`:

- Mong chờ: Nếu TCP thành công sẽ thấy Client Hello / Server Hello và tiếp tục là Application Data (HTTPS), hoặc thấy HTTP GET/200 (HTTP).
- Thực tế: Không có TLS/HTTP liên quan tới 10.255.255.1 hay 107.167.100.100.
- Ý nghĩa: Nếu không thấy TLS/HTTP cho IP đích thì TCP chưa tới tầng ứng dụng → kết nối không thành.

Khi dùng filter `tcp.flags.reset == 1` (RST packets):

- Mong chờ: Thường không mong thấy RST nhiều; có RST từ server nếu server chủ động reset kết nối (ví dụ không cho phép connection), hoặc từ firewall/router để chặn.
- Thực tế: Nhiều packet RST, ACK với source = 10.255.255.1 → destination = 192.168.0.237. Có 1 dòng ngược là 192.168.0.237 -> 162.247.243.29 [RST].
- Ý nghĩa: Khi RST đến từ IP 10.255.255.1 (IP mà đã map từ hosts), điều đó nghĩa là thiết bị ở IP đó đang chủ động reset kết nối — tức không chỉ drop im lặng mà phản hồi bằng RST. rằng khi client cố kết nối tới IP này thì kết nối bị chặn/đóng ngay lập tức bởi nguồn ở IP đó (hoặc một thiết bị đứng tên IP đó). Đây chính là nguyên nhân trực tiếp khiến người dùng không truy cập được.

No.	Time	Source	Destination	Protocol	Length	Info
156	4.732611	10.255.255.1	192.168.0.237	TCP	60	443 → 60074 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
164	4.984570	10.255.255.1	192.168.0.237	TCP	60	443 → 60078 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
171	5.258471	10.255.255.1	192.168.0.237	TCP	60	443 → 60074 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
176	5.498589	10.255.255.1	192.168.0.237	TCP	60	443 → 60078 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
181	5.773881	10.255.255.1	192.168.0.237	TCP	60	443 → 60074 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
187	6.009740	10.255.255.1	192.168.0.237	TCP	60	443 → 60078 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
193	6.286217	10.255.255.1	192.168.0.237	TCP	60	443 → 60074 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Hình 4.5: Reset

Nguyên nhân quan trọng nhất cho thấy vấn đề chính là ở RST packets: Người dùng không truy cập được vì khi kết nối tới IP mà domain được map tới (10.255.255.1), IP này reset kết nối (RST); kết quả là TCP handshake không thành, không có HTTP/TLS ứng dụng, do đó trình duyệt không load trang. Việc không thấy DNS query cho domain cho thấy tên đã bị ánh xạ cục bộ (hosts) — điều này giải thích vì sao client cố kết nối tới 10.255.255.1 thay vì IP thực của domain.

4.4.3 Nguyên nhân cốt lõi

- Khi client cố thiết lập kết nối TCP tới IP đích, kết nối bị từ chối/đóng ngay lập tức ở tầng TCP — thể hiện bằng TCP Reset (RST) từ địa chỉ phản hồi và/hoặc SYN retransmission do thiếu SYN-ACK. Nói cách khác: kết nối bị chặn hoặc host đích chủ động từ chối kết nối, nên TCP handshake không hoàn tất nên không có traffic ứng dụng (HTTP/TLS).
- Việc client nhận RST từ cùng một IP mục tiêu ngay sau khi gửi SYN cho thấy đích hoặc thiết bị trên đường tới đích chủ động chặn/không cho kết nối.
- Việc client cũng có nhiều SYN retransmission tới một số IP khác (không có SYN-ACK) cho thấy kết nối tới những IP này bị drop hoặc không có host phản hồi.

Về tổng thể: packet capture cho thấy mạng/TCP-level blocking (reset/drop) là nguyên nhân trực tiếp, chứ không phải vấn đề ở tầng ứng dụng (ví dụ lỗi HTTP).

4.4.4 Tìm và khắc phục lỗi

Trong trường hợp thực tế, ta sẽ thử thực hiện một vài bước như:

4.4.4.1 Xác định IP

Để biết chính xác IP đích:

- `ping www.example.com` - xem IP trả về (nếu không trả được, vẫn cho biết IP nếu DNS có).
- `nslookup www.example.com` - để biết DNS trả IP cho tên (thực tế).
- Wireshark: filter `dns` — tìm query/response cho domain.

4.4.4.2 Tìm nguồn RST / Nơi drop gói

Phân biệt middlebox (firewall/router) hay server từ chối

- `tcp.flags.reset == 1` — chọn một RST packet, mở packet details → Ethernet → xem Source MAC và phần IP → Source IP.
- Nếu Source MAC = gateway MAC (router nội bộ) → RST phát ra từ thiết bị mạng cục bộ.
- Nếu Source IP = server IP và Source MAC không phải gateway → server chính thức gửi RST.

4.4.4.3 Kiểm tra TCP connect

Biết được TCP bị reset, không phản hồi, hoặc thành công.

- `curl -v https://www.example.com` — xem chi tiết TCP/TLS response.
- `telnet <IP> 80` hoặc `telnet <IP> 443` — kiểm tra connect TCP (nếu telnet kết nối thì TCP OK).
- Wireshark filters: `tcp.analysis.retransmission, tcp.flags.reset == 1, tcp.flags.syn == 1 && tcp.flags.ack == 0`